Supplementary Document S1

Experimental protocol details for
"No finite rulebook can certify every safe program: an impossibility theorem for gatekeeping AI agents"

This document completes Appendix G and Section 6 of the article (version 14). It gives the register-machine semantics, derives the delay identities used in the designed program banks, describes the exact algorithms for the finite-state models, proves the soundness of the implemented certificate fragments, records notes on interpretation and reproducibility, describes the designed banks and their results (retained here; the article's Section 6 reports the random-population and self-referential experiments), details the exhaustive and sampled experiments, and, in Section S1.9, specifies the protocol of Simulations 1–5 of the article. Section, theorem and appendix numbers refer to the article.

S1.1 Register-machine transitions

Registers hold natural numbers. Validation checks instruction arity, nonnegative integer constants, register indices, jump destinations, and a valid fall-through successor for every nonterminal instruction other than an unconditional jump. A configuration comprises the program counter and all four registers. Table S1 gives the complete instruction semantics; registers not mentioned remain unchanged.

Table S1: Register-machine instructions. Execution starts at instruction 0 with registers $(n, 0, 0, 0)$. All nonjump, nonreturn instructions advance to the next instruction.

Instruction	Effect
`LOAD(i,k)`	Set $r_i = k$.
`MOV(i,j)`	Set $r_i = r_j$.
`INC(i)`	Increase r_i by one.
`DECJZ(i,l)`	If $r_i = 0$, jump to l; otherwise decrement r_i and advance.
`EQJUMP(i,k,l)`	If $r_i = k$, jump to l; otherwise advance.
`JMP(l)`	Jump to l.
`PUB(i)`	Append r_i to the public sequence.
`BAD`	Record the distinguished unauthorized event.
`RETURN(i)`	Terminate and return r_i.

S1.2 Delay identities

For a delayed event, loading takes one step, each of the d decrements and jumps takes two, the final zero test takes one, and the event takes one; the event therefore occurs at $2d + 3$. In the equal-output program with unequal delays, a branch, a load, $2d$ loop steps, the final zero test, a value load, and the publication take $2d + 5$ steps, whereas the immediate branch publishes at step 3. A delayed response adds a value load and a return after the final zero test, giving $2d + 4$, or 2052 when $d = 1024$.

S1.3 Exact finite-graph checks

For safety, the primary algorithm follows the deterministic trajectory until an event, termination, or repetition occurs. An independent reachable-state calculation checks whether any reachable state performs BAD. For response, the primary algorithm examines each initial trajectory. An independent least fixed-point calculation begins with all return states and repeatedly adds states whose successor already reaches a return state. Every input receives a response exactly when all three states belong to this fixed point.

For confidentiality, the next-public-event function skips internal transitions until a publication, termination, or repeated silent state. The primary algorithm follows pairs of next-public states and checks that their values match. An independent greatest fixed-point calculation begins with all state pairs and removes pairs in which only one side has a next event, the event values differ, or the successor pair has already been removed. The pair $(0, 1)$ remains exactly when the two sequences are equal. Both methods ignore delays and treat silent termination and silent divergence alike, as the norm requires.

S1.4 Soundness of the implemented certificate fragments

The certificate fragments report "permitted," "forbidden," or "uncertified." The last records the status of the method; it is not a third normative category.

For safety, an observed BAD is a finite witness of prohibition. Conversely, if the code contains no BAD instruction, execution cannot produce that event, which gives a sound syntactic certificate of permission.

For confidentiality, a value mismatch at a position present in both runs cannot be repaired by later publication. If one run has terminated and the other has published more values than that complete sequence contains, equal lengths are impossible. Both observations soundly certify prohibition. Two terminated, identical sequences certify permission. The input-independence test forbids every read of r_0, including comparisons, copying, increment, decrement, publication, or return. All other registers start equal across the two runs, and no allowed instruction transfers the initial secret to them. Induction over execution steps then gives identical control decisions and publications. Writes that discard r_0 preserve this sufficient condition.

For response, the control-flow certificate includes both outcomes of every conditional, and each concrete run follows this finite graph. If the reachable graph is acyclic and every terminal instruction returns, no run can continue indefinitely. Conversely, repetition of a complete nonterminal configuration forces the same future in a deterministic execution, so that input never receives a response. Growing counters do not repeat complete configurations and remain outside this fragment of forbidden certificates.

S1.5 Interpretation and reproducibility

Every label issued by these fragments is checked against the independently supplied bank labels. This checks the implemented examples, while the arguments above establish soundness beyond the bank. The fragments are not claimed to include every possible certificate.

The finite counts exhaust the defined class of 2744 models. The unbounded banks are deliberately designed examples, rather than random samples from a distribution of agents. A bounded test that fits a finite bank still does not establish universal closure, because larger delays and untested inputs already belong to the fixed general domain. The package records repeated execution and comparison of deterministic outputs. Runtime-version metadata may vary across Python installations and is excluded from the data comparison.

S1.6 Program banks

This section describes the designed banks that earlier versions of the article reported in full; the article now summarises them in one sentence of Section 6 and keeps them as an illustration of Appendix F. Supplementary Section S1.1 gives the instruction semantics, and Appendix F of the article proves that the lessons of the banks hold for every finite horizon.

Safety: events beyond the horizon.

For delays $d \in \{1, 2, 4, \ldots, 1024\}$, eleven programs load d, decrement it to zero, and then perform BAD. The first bad event occurs at transition $2d + 3$. Three safe controls return a constant, loop silently, or place an unreachable BAD after an unconditional return. Together, these programs form a bank of 14 candidates. Section S1.2 derives this and the other delay identities used below from the instruction semantics.

The bounded binary test permits a candidate if it observes no BAD by the horizon H. The sound partial method certifies prohibition when it observes a bad event and certifies permission when the code contains no BAD instruction. At $H = 4$, the binary test produces 11 false permissions, while the partial method leaves 12 candidates uncertified. At $H = 4096$, the binary test makes no errors on this bank, but the partial method still leaves the safe program with the unreachable event uncertified. Every certificate issued is correct. On the full domain, however, any finite H admits a larger delay satisfying $2d + 3 > H$, which defeats the bounded test.

Confidentiality: delay and missing publication.

This bank has 25 candidates. Eleven publish the secret after a delay. Eleven others publish equal outputs in their two runs, but after different delays. Two controls ignore the input. The last program publishes 0 when the secret is 0 but silently increments a counter forever when it is 1. Of these candidates, 13 are permitted and 12 forbidden. The delayed leaks produce the sequences [0] and [1] at transition $2d + 3$; the equal-output programs produce [0] at transitions 3 and $2d + 5$.

The bounded test permits a candidate exactly when the two prefixes observed by the horizon H are equal. At $H = 4$, it produces 11 false permissions and 11 false prohibitions. Equal empty prefixes can conceal future leaks, whereas prefixes of different lengths can later become equal complete sequences. The sound certificate fragment looks for four kinds of evidence: a value mismatch, an extra publication after the other run has terminated, two terminated identical sequences, or syntactic independence from the input register. It leaves 23 candidates uncertified at $H = 4$ and one at $H = 4096$, with no certificate errors. The remaining case, publication versus silence, is analytically forbidden. Its lack of a certificate reflects the limits of this fragment, not the unprovability of the individual case.

Response: coverage and eventual answers.

For each of eleven values $m \in \{1, 2, 4, \ldots, 1024\}$, two programs return 0 on every input except m. On input m, one of them repeats a state and the other increments a register forever. Two constant-return controls and one delayed but total response complete a bank of 25 candidates: 3 permitted and 22 forbidden. The delayed response returns after 2052 transitions on every input.

The binary test checks the inputs $0, \ldots, N - 1$ with horizon 64 and permits a candidate only if every tested run returns in time. At $N = 1$, it produces 22 false permissions and one false prohibition. At $N = 2048$, the tested inputs include every exceptional input in the bank, which eliminates false permissions, but the slow valid response remains falsely prohibited. The sound partial method certifies permission when the reachable control graph is acyclic and all terminal instructions return. It certifies prohibition when a tested input repeats a complete nonterminal configuration. It leaves 23 candidates uncertified at the initial setting and 12 at the final setting, with no certificate errors. Figure S1 summarizes the three banks.

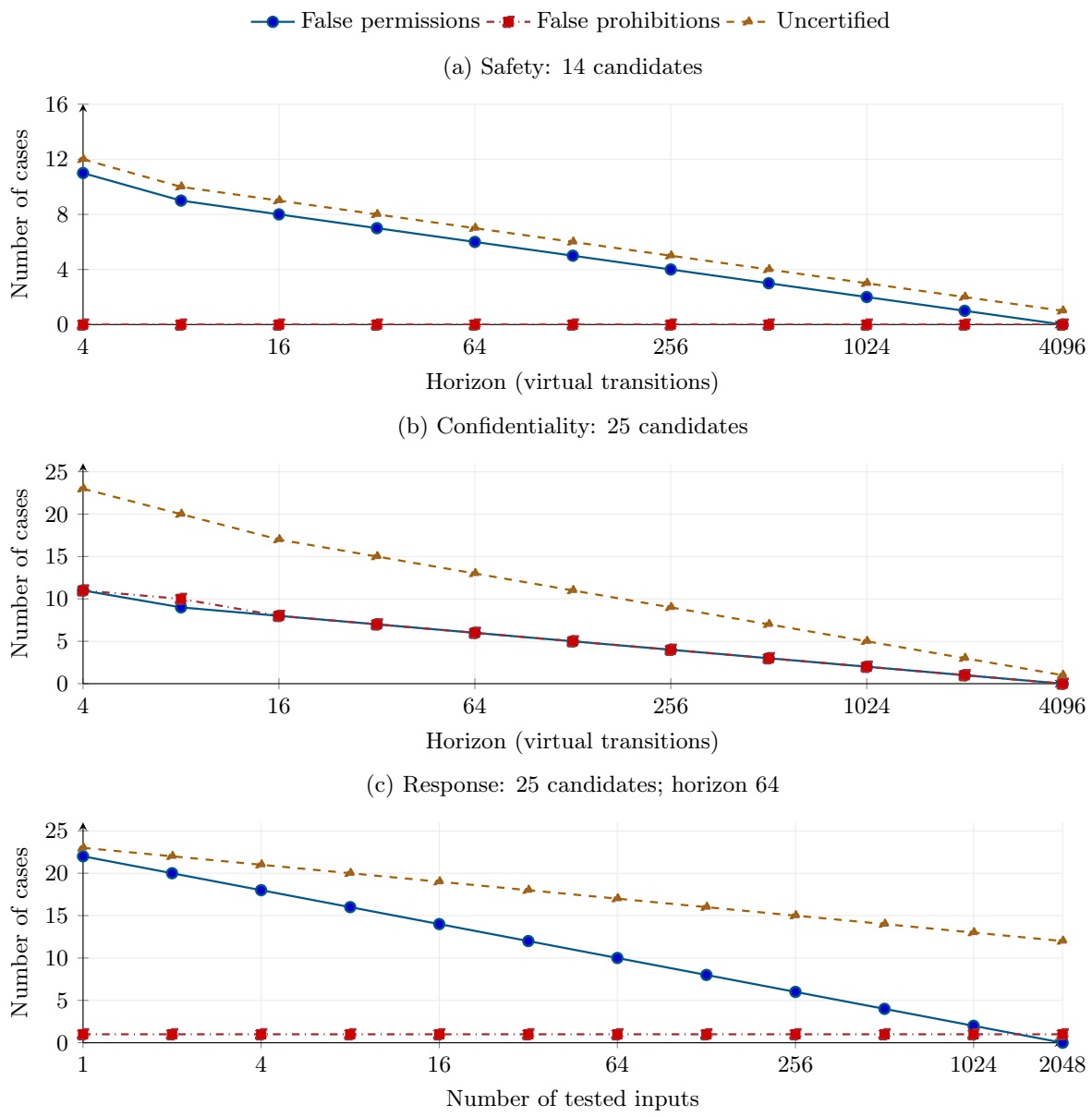


Figure S1: Bounded observations in the designed program banks. (a) Safety, 14 candidates. (b) Confidentiality, 25 candidates. (c) Response, 25 candidates with horizon 64. False permissions and false prohibitions count the errors of the bounded binary tests; uncertified counts describe the sound partial methods. Every issued certificate was correct. The uncertified count is not an error count.

S1.7 Additional exhaustive experiments

The script `run_extensions.py` produces panels A and B of Table G.1 of the article (Section 6, paragraphs "Exact classification of complete finite models" and "All short programs"). For the finite-state models, the checkable sets are: no state performs BAD (safety); no state publishes (confidentiality); and the successor graph has no cycle (response). For the short programs, every valid program of length one to four over registers r_0, r_1 and constants $0, 1$ is enumerated; the last instruction must be `RETURN` or `JMP`. The script re-implements the transition semantics and checks, on a sample of 970 runs, that every terminating run agrees with the original simulator.

Reference labels. Each run on inputs 0 and 1 is followed for at most 5000 steps before the stable-cycle check; safety refers to the run on input 0. A run is settled when it returns, when a complete configuration repeats, or when a stable-cycle certificate applies. That certificate continues the run, recording every event, until the program position repeats; it then executes one further pass of the resulting cycle and requires every conditional in it to be stable. A conditional is stable in three cases: it is an equality test on a register that the cycle never writes and never uses in a decrement test; it is an equality test on a register that the cycle only increases, never uses in a decrement test, and whose tested value already exceeds the constant; or it is a decrement test that read 0 on a register that no other instruction of the cycle writes. Every later pass then takes the same branches, so the run never leaves the cycle, and every later event is produced by an instruction of the cycle. The run is therefore safe if the cycle contains no BAD, and its recorded public sequence is complete if the cycle contains no publication; a cycle that publishes leaves confidentiality open. When the runs settle nothing, two syntactic rules apply: a program without a BAD instruction is safe, and a program that never reads its input is confidential. Both rules are sound under the declared semantics. With these rules every program is settled for safety, and 1,134 programs remain unresolved for confidentiality; unresolved programs are excluded from error counts. A timeout never defines a label.

Results. Among the 32,873 programs that contain a BAD instruction, 23,958 are forbidden and 8,915 permitted. At every horizon of 8 or more, the certificate method certifies all forbidden ones, by observing the operation, and none of the permitted ones. The latest first BAD event on input 0 occurs at step 6. At horizon 1024, all 918 false prohibitions for confidentiality involve two runs that publish the same periodic sequence on different schedules. The first publications differ by 0 steps in 517 programs, by 1 in 286, by 2 in 75, by 3 in 34, by 4 in 4, and by 5 in 2. Even when the first publications coincide, later ones occur at different times, so prefixes observed at a fixed horizon differ in length. Table S2 lists the results at every horizon.

S1.8 Sampled longer programs

The script `run_sampled.py` (Appendix G of the article; panel C of Table G.1) draws 50,000 programs of each length 6, 8, 10, and 12 with seed 20261013 and applies the reference labels, bounded tests, and certificate methods of Section S1.7 unchanged. Reference labels (permitted / forbidden / unresolved): length 6: safety 49,055 / 945 / 0, confidentiality 48,689 / 1,200 / 111; length 8: safety 49,031 / 969 / 0, confidentiality 48,595 / 1,226 / 179; length 10: safety 49,044 / 954 / 2, confidentiality 48,548 / 1,294 / 158; length 12: safety 49,162 / 838 / 0, confidentiality 48,420 / 1,410 / 170. The latest first forbidden operation on input 0 occurs at steps 7, 9, 17, 27 for the four lengths; the numbers of forbidden programs whose first forbidden operation occurs after step 8 are 0, 1, 32, 71, and every one of them contains a backward jump. Table S3 lists the results at every horizon.

Table S2: All 483,641 short programs: results at every horizon H. Reference labels: safety, 459,683 permitted, 23,958 forbidden, none unresolved; confidentiality, 452,223 permitted, 30,284 forbidden, 1,134 unresolved. False prohibitions do not occur for the safety test. Certified counts refer to the sound certificate methods; no certificate was wrong.

Requirement	H	False permissions	False prohibitions	Certified
Safety	4	22	0	474,704
Confidentiality	4	613	756	203,577
Safety	8	0	0	474,726
Confidentiality	8	0	1,210	205,343
Safety	16	0	0	474,726
Confidentiality	16	0	920	205,343
Safety	32	0	0	474,726
Confidentiality	32	0	1,224	205,343
Safety	64	0	0	474,726
Confidentiality	64	0	918	205,343
Safety	128	0	0	474,726
Confidentiality	128	0	1,222	205,343
Safety	256	0	0	474,726
Confidentiality	256	0	920	205,343
Safety	512	0	0	474,726
Confidentiality	512	0	1,224	205,343
Safety	1024	0	0	474,726
Confidentiality	1024	0	918	205,343

Table S3: Sampled programs: results at every horizon H, by length. False prohibitions do not occur for the safety test. Certified counts refer to the sound certificate methods; no certificate was wrong.

Length	Requirement	Quantity	H=4	H=8	H=16	H=32	H=64	H=128	H=256	H=512	H=1024
6	Safety	false permissions	132	0	0	0	0	0	0	0	0
6	Safety	certified	49,601	49,733	49,733	49,733	49,733	49,733	49,733	49,733	49,733
6	Confidentiality	false permissions	196	6	0	0	0	0	0	0	0
6	Confidentiality	false prohibitions	41	35	46	45	51	44	48	45	51
6	Confidentiality	certified	12,585	19,467	19,498	19,505	19,505	19,505	19,505	19,505	19,505
8	Safety	false permissions	263	1	0	0	0	0	0	0	0
8	Safety	certified	49,361	49,623	49,624	49,624	49,624	49,624	49,624	49,624	49,624
8	Confidentiality	false permissions	430	36	0	0	0	0	0	0	0
8	Confidentiality	false prohibitions	62	50	63	64	75	73	74	65	77
8	Confidentiality	certified	7,365	13,162	13,261	13,267	13,269	13,269	13,269	13,269	13,269
10	Safety	false permissions	368	32	1	0	0	0	0	0	0
10	Safety	certified	49,148	49,484	49,515	49,516	49,516	49,516	49,516	49,516	49,516
10	Confidentiality	false permissions	571	111	4	2	0	0	0	0	0
10	Confidentiality	false prohibitions	63	71	92	97	100	99	103	94	104
10	Confidentiality	certified	4,645	7,396	9,708	9,726	9,727	9,727	9,727	9,727	9,727
12	Safety	false permissions	362	71	1	0	0	0	0	0	0
12	Safety	certified	49,079	49,370	49,440	49,441	49,441	49,441	49,441	49,441	49,441
12	Confidentiality	false permissions	772	228	10	2	0	0	0	0	0
12	Confidentiality	false prohibitions	58	105	114	133	133	141	128	142	133
12	Confidentiality	certified	3,246	5,496	7,624	7,657	7,663	7,663	7,663	7,663	7,663

S1.9 Simulations 1–5 of Section 6

Simulations 1, 3 and 5 use a *step-counted environment* written in Python, which serves as the universal environment of Section 2 of the article; it is defined in the script `sim1_diagonal_gatekeeper.py`. A plan is a source text defining a generator `run(h, env)`; each `yield` is one step. The environment provides `env.bad()` (the forbidden operation), `env.pub(v)` (publication) and `env.sim(src, h, T)`, which runs another plan internally for at most T steps and yields once outward for every inner step, so that simulation costs one step per simulated step (the linear overhead $g(t) = t$ of the assumption of Section 4). Internal simulations never reach the external channels. A *gatekeeper* is a source text defining a generator `check(src, T, env)` that returns `True` (permitted: no BAD within T steps) or `False`; its cost is the number of steps it yields. Ground truth for a plan and budget T is obtained by running the plan at top level for T steps.

Simulation 1 (adversary generator). Given a gatekeeper's source and a budget T, the generator emits the plan d_T of Theorem 3: a template that contains the gatekeeper's source, T, and its own template text, reconstructs its complete source by substitution (a quine), calls `env.check` on itself, and performs BAD if the verdict is "permitted" and returns silently otherwise. The script asserts that the reconstructed source equals the emitted one. Eight gatekeepers are tested at $T = 8, 16, \ldots, 1024$: simulate for T, $T/2$ and $\lfloor\sqrt{T}\rfloor$ steps; permit iff the text `bad(` is absent; always permit; always forbid; a seeded coin flip; and permit iff the source is shorter than 400 characters. A benign benchmark of five plans with known truth (silent loop; quick return; BAD at step 3; BAD at step $T - 1$; BAD after the budget) is also scored. Outputs: `sim1_adversary_results.csv` (verdict, truth, cost, step of the first BAD) and `sim1_benign_benchmark.csv`.

Simulation 2 (horizon–length landscape). `sim2_horizon_landscape.py` draws, for each length $L \in \{4, 6, \ldots, 24\}$, 200,000 programs with the instruction options of Section S1.8 over registers r_0–r_2 and constants 0–7 (seed $20261020 \cdot 1000 + L$ per length), and applies the reference labels, bounded tests and certificate methods of Section S1.7 unchanged, imported from `run_extensions.py`, at horizons $H = 4, \ldots, 4096$. Outputs: `sim2_landscape.csv` (per length and horizon: label counts, false permissions, false prohibitions, certified counts) and `sim2_first_bad_meta.json` (histogram of the first forbidden step per length), from which `sim2_required_horizon.csv` derives the percentiles. Table S4 lists the per-length results.

Simulation 3 (open-problem plans). `sim3_open_problem_programs.py` runs three plans in the step-counted environment at horizons $10, 10^2, \ldots, 10^6$: the Goldbach search (one step per even number examined; BAD if an even number ≥ 4 is not a sum of two primes), the Collatz search (one step per starting value; BAD if a trajectory exceeds 10^7 iterations without reaching 1), and a control encoding the false statement "every square is below one million" (BAD at $n = 1001$). Output: `sim3_open_problem_results.csv`.

Simulation 4 (what the plateau is made of). `sim4_plateau_breakdown.py` regenerates the Simulation 2 population and, for every compliant program wrongly forbidden by the bounded confidentiality test at $H \in \{64, 1024, 4096\}$, records whether both runs halted and whether each run's public form is an infinite periodic sequence; it also counts the compliant programs whose two runs both halt within H and the test's errors on them. Output: `sim4_plateau_breakdown.csv`.

Simulation 5 (rulebook-sized omissions). `sim5_rulebook_sized_omission.py` builds, for each of four sound safety certifiers written as step-counted gatekeepers (the text contains no `bad(`; no loops and no `bad(`; the plan halts within 1,000 simulated steps without BAD; `bad(` occurs only after an unconditional `return` at the same indentation), the plan q of Proposition 3.10: it reconstructs its own source, asks the certifier about itself, performs BAD if certified, and

otherwise idles forever. The script checks that each certifier is sound and non-trivial on a three-plan benchmark, that q is not certified, that q performs no BAD within 10^5 steps, and records $|q| - |\text{certifier}|$. Output: `sim5_rulebook_sized_omission.csv`.

Reproducibility. Every script is deterministic; Simulations 2 and 4 use `multiprocessing` over lengths and take about 30 and 25 seconds on ten cores, Simulation 3 about 70 seconds (dominated by the Goldbach run to 10^6 steps), Simulations 1 and 5 about one second each. The figure scripts `figures/build_fig1_data.py`, `build_fig3_data.py` and `build_fig4.py` read these CSV files and refuse to write a figure whose printed claim the data contradict.

Table S4: Simulation 2 by program length (200,000 programs per length; registers r_0–r_2, constants 0–7; seed 20261020). Unsafe: programs forbidden under the safety norm. Unresolved: programs excluded from error counts (safety / confidentiality). False permissions: share of unsafe programs permitted by the bounded safety test at horizon H. Plateau: share of compliant programs wrongly forbidden by the bounded confidentiality test at $H = 4096$. First violation: median / 99th / 99.9th percentile / maximum of the step at which an unsafe program first performs BAD on input 0. Values are computed from `sim2_landscape.csv` and `sim2_required_horizon.csv`.

L	Unsafe	Unresolved	False permissions (%) at $H =$				Plateau (%)	First violation
			8	16	32	64		
4	3,337	0 / 100	0.00	0.00	0.00	0.00	0.029	2 / 3 / 4 / 5
6	3,752	0 / 468	0.05	0.00	0.00	0.00	0.107	3 / 5 / 7 / 11
8	3,536	1 / 649	0.06	0.00	0.00	0.00	0.166	3 / 7 / 8 / 9
10	3,602	1 / 731	3.89	0.00	0.00	0.00	0.228	4 / 9 / 11 / 13
12	3,413	7 / 701	9.61	0.09	0.00	0.00	0.265	4 / 11 / 13 / 26
14	3,312	5 / 674	14.70	0.15	0.09	0.03	0.322	4 / 13 / 17 / 66
16	3,138	9 / 668	18.10	0.45	0.03	0.00	0.326	5 / 15 / 20 / 52
18	3,115	9 / 626	21.25	1.16	0.16	0.00	0.307	5 / 17 / 39 / 60
20	2,938	10 / 626	25.63	1.80	0.14	0.03	0.344	5 / 18 / 36 / 123
22	2,744	3 / 573	26.49	2.33	0.04	0.00	0.368	5 / 18 / 28 / 55
24	2,756	2 / 504	30.15	2.94	0.22	0.00	0.338	6 / 21 / 33 / 47